

CacheWise: Understanding Workloads and Optimizing KVCache Management for Efficiently Serving LLM Coding Agents

Shubham Tiwari
University of Washington
Seattle, WA, USA

Simon Peter
University of Washington
Seattle, WA, USA

Tapan Chugh
University of Washington
Seattle, WA, USA

Ratul Mahajan
University of Washington
Seattle, WA, USA

Nash Rickert
University of Washington
Seattle, WA, USA

Haiying Shen
University of Virginia
Charlottesville, VA, USA

Abstract

Coding agents are a fast-growing LLM application, executing as long-running closed-loop sessions in which LLM generations alternate with external tool calls. Yet, unlike chat workloads, their serving behavior has not been studied extensively. We address this gap by collecting a dataset of real-world coding assistant traces. Our analysis shows that coding agent sessions repeatedly reuse large prefixes and create sustained KVCache pressure that conventional LLM serving policies handle poorly.

Based on our analysis, we present CACHEWISE, a KVCache management layer that improves KVCache reuse for coding agent workloads. CACHEWISE combines prefix-aware scheduling with reuse-aware eviction guided by lightweight predictions from tool call metadata. Implemented in vLLM and evaluated on the collected traces, CACHEWISE reduces KVCache evictions by up to 2–2.6× and improves total agent session completion time by up to ~3.5×. CACHEWISE is open sourced at github.com/cachewise-project/cachewise-coding-traces

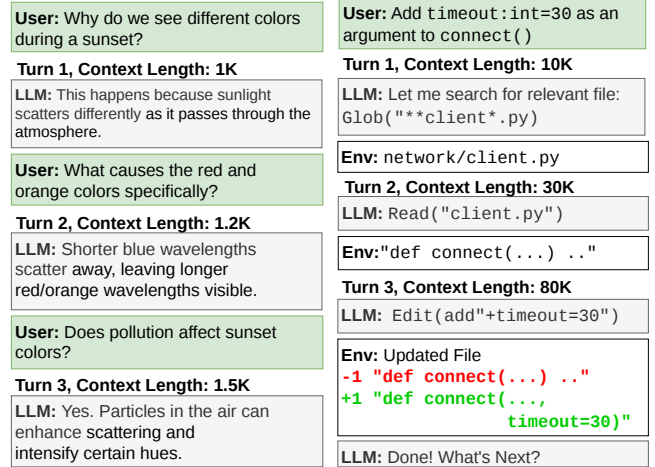
1 Introduction

Coding assistance [5, 16, 25], where a Large Language Model (LLM) generates code based on user instructions, is emerging as a crucial application with significant potential. Unlike chatbots (Figure 1b), where the user sends a query and the LLM generates a corresponding response, coding agents [19, 39, 44] formulate multi-step plans and execute them through a series of tool calls [27, 41] to run code, inspect program outputs, or modify files (Figure 1c). As Figure 1a shows, the execution alternates between the LLM, which processes input and generates function calls, and the *environment*, which executes those calls and performs actions such as reading/writing code and running tests.

While LLM serving in the context of chat systems has been studied extensively [3, 14, 21, 26, 28, 35, 42, 45, 46], coding agent serving systems have not been examined previously, to the best of our knowledge. A key reason is the lack of publicly available real-world traces for this workload. To address this



(a) Coding agent workflow.



(b) Chat session.

(c) Coding assistant session.

Figure 1. Chat and coding-assistant interactions.

gap, we collect a dataset of real-world coding assistant traces from researchers actively using coding assistants for research.

In this work, we use this collected dataset to study coding agent workloads and their implications for serving system design. Compared to chatbot workloads, coding-agent sessions are long-running, accumulate substantially larger contexts, and are dominated by tool-initiated turns rather than direct user input. A single user task therefore expands into a closed-loop sequence of LLM requests and tool calls, where each new turn reuses and extends a growing prefix. Consequently, coding assistance is fundamentally a *session-oriented* serving workload: the key objective is not the latency of an individual request, but the completion time of the full multi-turn session.

Existing LLM serving systems are primarily designed and evaluated for prior workloads such as chatbot applications. Their default scheduling and KVCache management policies

are not well-suited for coding agent workloads. In particular, systems such as vLLM [21] and Mooncake [28] employ First-Come-First-Served (FCFS) batching and Least-Recently-Used (LRU) KVCache eviction. These workload-agnostic policies ignore two defining properties of coding-agent sessions: requests from the same session frequently have high prefix overlap with already-resident state, and the time to next reuse depends strongly on the ongoing tool execution. FCFS interleaves many sessions and expands the active KVCache working set, increasing the likelihood of evicting prefixes that will soon be needed again. LRU, in turn, uses only past access recency and cannot distinguish between a session whose tool call is about to complete and one whose state will remain unused for much longer. Under memory pressure, these policies therefore trigger KVCache thrashing, reducing useful work and lowering token goodput (i.e., the rate of useful token processing after excluding recomputation and KVCache movement overheads). While some recent works [15, 23] attempt to optimize serving for agentic workloads, they tightly couple the agent implementation framework with the LLM serving system, and thus are ill-suited for large-scale deployments, which must support diverse clients.

To address these challenges, we introduce CACHEWISE, an agent-aware KVCache management layer that reduces the overhead from KVCache recomputation and offload for general-purpose LLM serving systems. CACHEWISE is built on two key ideas: prefix-aware request scheduling and reuse-aware KVCache eviction. Prefix-aware request scheduling prioritizes inference requests with a higher degree of prefix overlap with KVCache state already maintained in accelerator memory. By doing so, it reduces the number of blocks that must be reclaimed and later restored. Complementing this, reuse-aware KVCache eviction evicts prefixes based on anticipated reuse likelihood, rather than relying purely on access recency. Notably, CACHEWISE leverages session metadata, such as tool execution information, as signals to predict the reuse likelihood of KVCache prefixes using lightweight predictors trained on historical samples collected by the serving system, without any modifications to the coding agents themselves. This enables, CACHEWISE to sustain high accelerator utilization while reducing the frequency of KVCache evictions, thereby improving overall token goodput, i.e., the rate at which new LLM tokens are generated.

This paper makes the following contributions:

- We collect and analyze real-world coding agent traces, and use them to characterize coding agents as a distinct class of LLM serving workloads. We contrast them with traditional chatbot and other multi-turn tool-use workloads, identify the systems implications of closed-loop execution, long-lived sessions, and large, growing prefixes. We have open sourced the dataset at github.com/cachewise-project/cachewise-coding-traces.

- We design and implement CACHEWISE, based on two key ideas: prefix-aware scheduling and predictive KVCache eviction. We implement our techniques on top of vLLM [21].
- We evaluate our techniques using real-world coding agent traces. Our results show that CACHEWISE reduces KVCache evictions by 2–2.6 $\times$ compared to state-of-the-art systems such as vLLM, significantly improving token goodput. In addition, CACHEWISE improves total request completion time by up to $\sim 3.5\times$ and achieves performance comparable to a predictive eviction policy with ground-truth reuse information.

2 Background: Efficient LLM Serving

LLM serving typically requires expensive and scarce accelerators such as GPUs and TPUs (we will collectively call them XPU). Efficiently using their massive internal resources requires massively parallel operations to optimize their internal compute and memory resources. Although serving requests consist of separate *prefill* and *decode* stages, which require fundamentally different optimization strategies, efficient XPU memory management is crucial for both.

During prefill, serving requires a large number of parallel compute operations to process the input sequence and generate the intermediate key-value tensors (KVCache) corresponding to each input token. During decode, efficient serving systems typically employ *continuous batching* [21, 42] to ensure parallelism *across requests*, since the LLM only generates output tokens sequentially, due to the autoregressive nature of the Transformer architecture [36]. While parallelism within a sequence (for prefill) or across multiple sequences, through batching [42] (for decode) improves XPU efficiency, the benefits are limited by the XPU memory available to store the KVCache states of all the requests in the batch.

Similarly, the available KVCache memory is a bottleneck for optimizing the prefill phase of serving requests that share a common prefix. Although state-of-the-art serving systems, e.g., vLLM [21], SGLang [45], persist KVCache from previous requests to minimize the number of operations required for executing requests that extend a previous request, during memory pressure, persisted KVCache prefixes must be evicted to allocate memory for requests whose prefixes are not already available. If a subsequent request arrives whose prefix corresponds to evicted KVCache state, the prefix must be rematerialized which either requires recomputing the KVCache or transferring it back from lower storage tiers, such as host DRAM, SSD, or remote memory, if available (e.g., LMCACHE [10], Mooncake [28]). Notice that although some systems, e.g., Infercept [2], make KVCache management decisions per-request, state-of-the-art serving systems typically implement a block-based abstraction, analogous to memory paging in operating systems, that partitions the KVCache memory pool in the XPU into fixed-size blocks, and eviction decisions can be made at per-block to reduce wastage.

Trace Captures?	CATraces (ours)	SWE-Agent[40]	T1 [9]	Toucan [38]	ShareGPT [29]
# Tokens	10M	474M	2.3K	0.94M	0.26M
Workload Domain	Coding Assistant	Software Engg.	Tool Usage	Tool Usage	Chat
Interactive Sessions?	✓	✗	✗	✗	✓
Real Tasks?	✓	✗	✗	✗	✓
Real Tools?	✓	✓	✗	✓	✗

Table 1. Comparison of coding agent and multi-turn datasets.

3 Coding Agent Workloads

To understand the system implications of coding agent workloads, we present a detailed characterization of traces collected from users of Claude Code [5], a popular coding assistant application, performing real-world software development and engineering tasks on various open source projects.

Our collected dataset¹, denoted as CATraces, includes detailed conversations: user instructions, model outputs, tool calls, their results, etc., and annotated with key system metadata e.g., number of tokens, timestamps for each message, human interventions, and so on. Table 1 highlights the unique aspects compared to previously available LLM workload traces. To the best of our knowledge, we are the first to present the real-world usage of coding assistant workloads; ShareGPT [29] presents real-world interactions for chat workloads, while others usually contain synthetic tasks [9, 19, 40], sometimes against mock tools [38].

CATraces Overview: We begin by characterizing structural differences between real-world coding agent workloads and prior datasets. First, observe in Figure 3, that coding agent serving requests have orders of magnitude more *turns* than other workloads, where a turn refers to a new LLM request building upon a previous request and result. Notice that each turn presents an opportunity for KVCache reuse; if the entire accumulated prefix needed to be recomputed for each turn, the overall number of flops required to compute would be orders to magnitude higher. Second, observe in Figure 2 that the distribution of the number of tokens to prefill and decode per request is significantly different for coding agent workloads compared to others: without KVCache sharing across turns, each request must prefill a substantially higher number of tokens, whereas each generates significantly fewer decode tokens, leading to $\sim 21\times$ higher ratio of prefill to decode tokens compared to chatbot workloads. These differences have crucial implications for future systems [12, 43]: the significantly larger shared KVCache prefixes implies that only fewer KVCache blocks can be stored in XPU memory concurrently, which reduces the number of requests that can be executing concurrently, and constrains the overall token generation throughput during the decode phase (§2). Overall, optimizing KVCache management is especially crucial for serving

coding agent workloads efficiently.

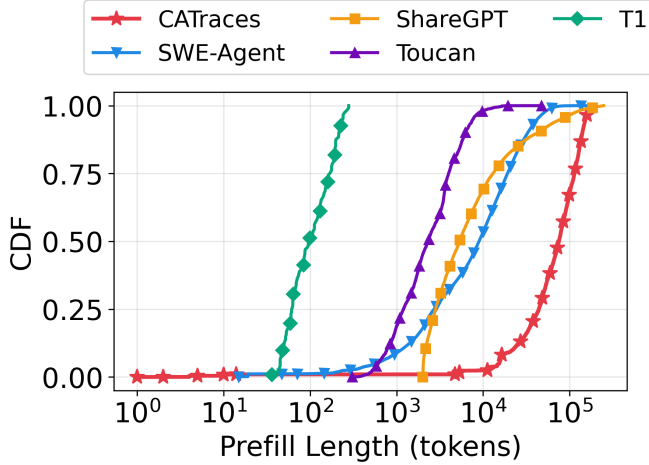
Closed-loop execution shifts performance objective: We analyze the distribution of requests initiated by tool completion versus those initiated by direct user input. Figure 4 shows this distribution. We observe that requests triggered by tool completion (i.e., model-generated tool calls) [33, 41] are $20\times$ more frequent than user-initiated requests at the median, indicating that the request generation process is predominantly closed-loop. Figure 1 illustrates this behavior: while chat workloads consist of simple user-LLM interactions, coding agent workloads form closed-loop chains where a single user request triggers a sequence of LLM requests and tool calls. To evaluate the performance of coding agent workloads, session-level metrics (such as session completion time) capture the system’s efficiency in advancing multi-step execution within a session, whereas widely used token-level metrics such as Time-To-First-Token (TTFT) and Time-Between-Tokens (TBT) capture only per-request latency.

Long-running sessions compete for limited accelerator memory: We analyze session durations by computing the time difference between the first and last LLM request in each session of CATraces. Figure 5 shows the distribution of session durations. We observe substantial variation in duration, with long-running sessions being common (e.g., 36 min at median, >2.6 hours at tail). Such long-running sessions imply that KVCache state must persist in XPU memory over long periods to enable reuse across turns. Each turn of the session appends additional context to the prefix, causing the KVCache memory to grow over time (as shown in Figure 6), and remain resident for the duration of the session. As multiple long-running sessions coexist, their KVCache state collectively competes for limited XPU memory. Large KVCache prefixes directly translate to higher eviction overheads. Evicting large KVCache states requires recomputation of state or data transfer across memory tiers, both of which incur significant overhead and impact system’s efficiency.

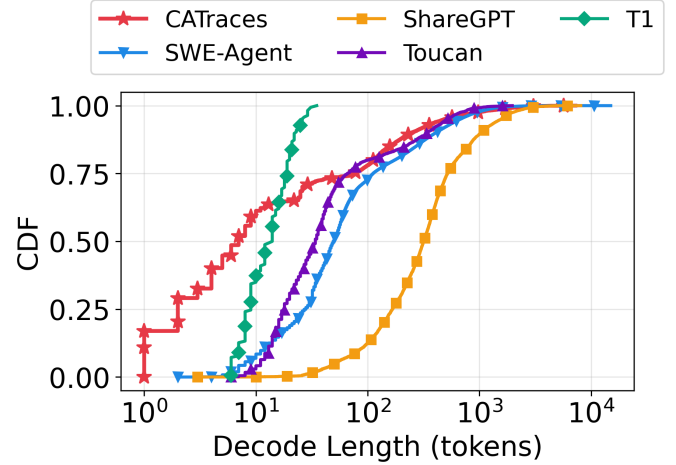
Inter-request time varies, but carries meaningful structure across toolcalls: We analyze the types of tools invoked and their execution times in CATraces. Figure 8 shows that tool execution times vary significantly across tool types and exhibit long-tailed behavior. For example, `grep` has short and tightly clustered durations, whereas `bash` exhibits longer and more variable execution times. This variability translates directly into large fluctuations in the time between successive serving requests from the same session. Figure 7 shows that tool executions induce substantial variation in inter-arrival times, leading to irregular reuse intervals for KVCache state. Eviction policies that rely solely on past access patterns (e.g., LRU) may evict state that is about to be reused, resulting in unnecessary evictions.

Despite variations, tool call durations are not arbitrary. Figure 9 shows that tool metadata, such as tool type and arguments, captures meaningful structure in execution behavior.

¹We collected anonymized traces from consenting participants within our lab. This data collection follows standard ethical guidelines for research.



(a) Prefill length distributions for different datasets



(b) Decode length distributions for different datasets

Figure 2. Comparing LLM serving request characteristics across datasets.

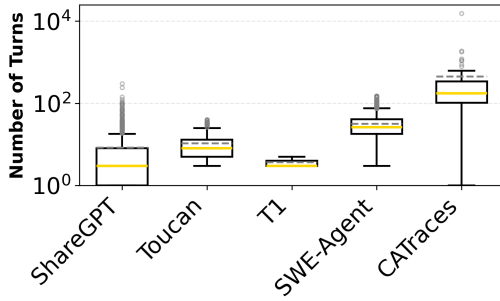


Figure 3. Distribution of number of turns for different workloads.

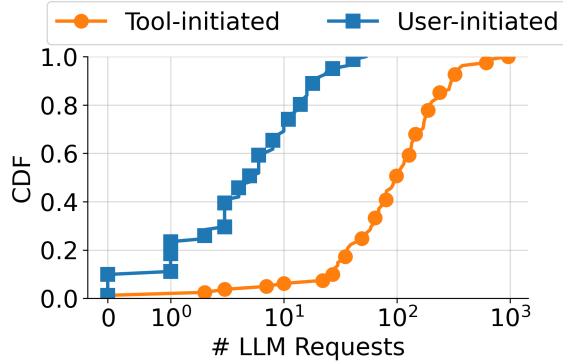


Figure 4. CDF of requests triggered by tool completion versus user input. Tool-initiated requests dominate, indicating a predominantly closed-loop request generation process.

For instance, simple file-system operations (e.g., `ls`, `grep`) have short durations, whereas more complex operations (e.g., `pytest`) exhibit significantly longer execution times. This observation exposes opportunities for KVCache optimization by leveraging tool metadata as signals for reuse prediction, which we discuss further in §5.

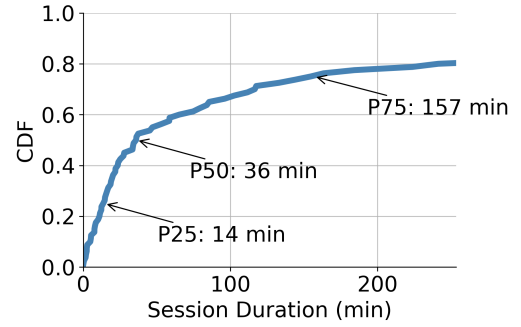


Figure 5. Session duration distribution observed in CATraces.

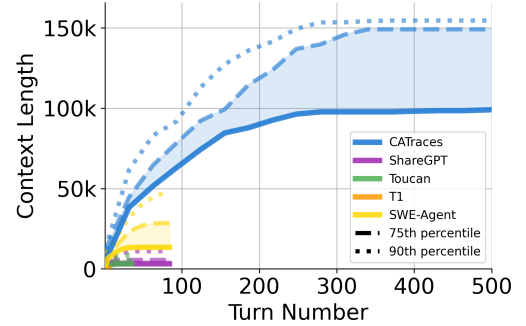


Figure 6. Context Length growth in number of tokens.

4 Implications for Efficient LLM Serving

Coding agent workloads exhibit strong temporal locality, where successive requests within a session share and reuse substantial portions of previously computed KVCache state. Existing LLM serving systems manage KVCache allocation and eviction independently at the granularity of individual requests, without accounting for future reuse across successive requests within a session. This mismatch leads to significantly lower KVCache reuse and system efficiency.

We introduce a lightweight formal model to precisely characterize the resource dynamics of KVCache management in

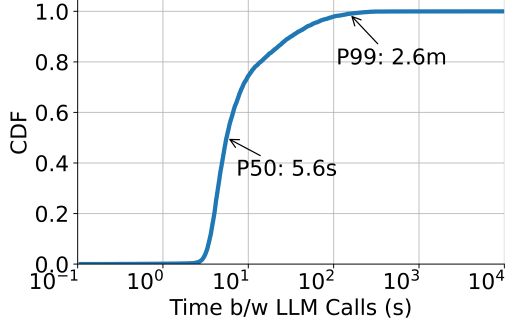


Figure 7. Time between consecutive LLM requests.

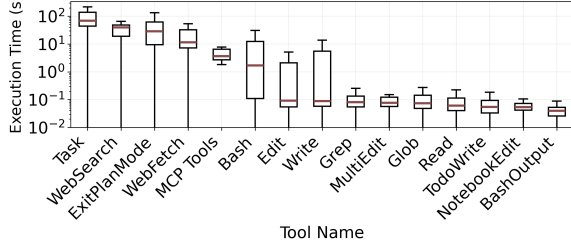


Figure 8. Distribution of tool execution durations.

```
{
  "tool": "bash",
  "args": "ls -la",
  "duration_ms": 49
}
{
  "tool": "bash",
  "args": "cd vllm && git
log --all -p --grep
='SchedulingPolicy'
-- '*.py' | head -300
",
  "duration_ms": 1143
}
{
  "tool": "bash",
  "args": "uv run --
frozen pytest tests/
client
/test_set_roots.py -xvs
",
  "duration_ms": 83333
}
{
  "tool": "Grep",
  "args": {"path": "vllm/
vllm", "pattern": "
start_load_kv",
  "duration_ms": 183
}
{
  "tool": "Glob",
  "args": "**/scheduler*.
py",
  "duration_ms": 147
}
{
  "tool": "WebFetch",
  "args": "https://
gofastmcp.com
/servers/context",
  "duration_ms": 39988
}
```

Figure 9. Examples of tool invocations observed in CATraces.

LLM serving and highlight the implications for coding agents. Figure 10 illustrates the memory layout that motivates our notation.

Let M denote the total KVCache block capacity of an inference node (i.e., XPU memory available for KVCache after accounting for model weights and runtime overhead). At any time t , let $S_t = \{S_1, S_2, \dots\}$ be the set of *active sessions* co-located on the node. Each session S_i has a contiguous, monotonically growing prefix of KVCache blocks d_i , out of which $k_i(t)$ are resident in XPU memory at time t . The *active KVCache working set* is $W(t) = \sum_{i \in S_t} d_i$, and the system operates within memory budget (requires no eviction) as long

as $W(t) \leq M$.

When session S_i issues a new request r_i at time t , it requires d_i KVCache blocks in total to serve the full context. Of these, $k_i(t)$ blocks already reside in memory (the *reusable prefix overlap*), so the number of *additional* blocks that must be allocated is $a_i(t) = d_i - k_i(t)$. If $W(t) + a_i(t) \leq M$, the request is admitted immediately. Otherwise, the system must *evict* $a_i(t)$ blocks from one or more other sessions before admission.

Implication #1: FCFS scheduling thrashes KVCache of long-running agent sessions: At any given time t , a session S_j can be active (i.e., executing a request on the XPU), queued (i.e., request is waiting to be executed due to insufficient XPU memory), or inactive (i.e., waiting upon user or the environment to submit the next request). Existing systems which admit requests in FCFS order interleave requests from multiple queued sessions, which attempts to increase k_i for many different sessions. As the working set grows, it competes for limited XPU memory and can exceed the available HBM capacity. When this occurs, KVCache state from one session is evicted to accommodate requests from other sessions. Since this evicted state is often reused by subsequent requests within the same session, FCFS scheduling leads to repeated eviction and recomputation (or movement) of state, resulting in KVCache thrashing.

Implication #2: LRU eviction does not account for future KVCache reuse when toolcalls return: When $W(t) + a_i > M$, the system faces a trade-off: admitting r_i requires evicting blocks whose τ_j may be small, incurring recomputation overhead and reducing system goodput. However, not all evictions are equally costly. Let τ_i denote the *time to next reuse* of session S_i 's resident blocks, i.e., the time until S_i issues its next request and those blocks are accessed again. Evicting blocks from a session with small τ_i forces an imminent recomputation (or data movement) penalty; evicting from a session with large τ_i is comparatively cheap. An ideal eviction policy therefore selects $j^* = \arg \max_{j \in S_t, j \neq i} \tau_j$, retaining sessions whose KVCache will be needed soonest.

In-practice, when serving coding agents with multiple long-running, KVCache working set is likely to exceed the available XPU memory. Since τ_i is unknown, a workload-agnostic policy (e.g., LRU) resolves this tension using only past access recency, which can lead to priority inversions and blocks with high reuse value are evicted and subsequent requests from the same session require these blocks to be recomputed or moved back into memory. Repeated eviction and restoration of state leads to high overhead in recomputation and data movement.

5 CACHEWISE Design

Drawing from our measurements and analysis, we present CACHEWISE (Table 2). Unlike existing systems that optimize KVCache management for individual request processing commonly found in chatbots, CACHEWISE maximizes KVCache

Property	Workload Characteristic	Baseline Limitation	CACHEWISE Design Choice
Closed-loop sessions	User tasks trigger multi-turn sessions of LLM calls and tool executions.	Per-request metrics (TTFT, TBT) do not capture session-level efficiency.	Optimize for end-to-end session completion for closed-loop workloads.
Expanding KV-Cache prefixes	Sessions accumulate large KVCache prefixes that compete for limited XPU memory.	FCFS request scheduling disregards KV-Cache residency which leads to thrashing across sessions.	Prefix-aware scheduling forms batches in priority order to maximize KVCache reuse.
Variable inter-request times	Tool execution durations vary by orders of magnitude due to different tool types and arguments.	LRU eviction uses only access recency that which leads to priority inversions w.r.t. future re-use.	Predictive eviction estimates the order of future reuse from tool metadata to guide eviction decisions.

Table 2. Summary of key findings. Each row identifies a characteristic of coding agent workloads, the limitation it exposes in existing systems, and the corresponding CACHEWISE design choice.

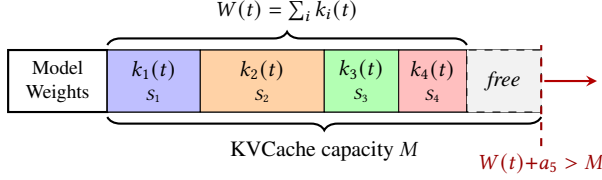


Figure 10. XPU memory layout during LLM serving. After reserving memory for model weights, the remaining capacity M holds KVCache blocks for active sessions $S_i = \{S_1, \dots, S_4\}$. The working set $W(t) = \sum_i k_i(t)$ grows as sessions accumulate context. When a new request r_5 requires a_5 additional blocks that exceed M , the system must evict blocks from an existing session.

reuse across requests in long-running, closed-loop agent sessions (e.g., coding agent workloads). Higher KVCache reuse reduces the eviction overhead (recomputation and data movement overhead of KVCache across storage tiers) and allows executing more sessions concurrently which improves the end-to-end session performance and the system’s token goodput.

Workflow: Figure 11 (A) illustrates the end-to-end workflow of a session in an LLM serving system with CACHEWISE integrated at the inference node. Incoming requests from active sessions are routed by the load balancer to an inference node. Since existing load balancers typically use prefix-match based routing [1, 8, 11], i.e., a request is routed to the inference node with largest reusable KVCache prefix resident in the XPU memory, requests from the same session, which share the growing KVCache prefix, are likely to be routed to the same inference node. CACHEWISE optimizes KVCache management at each inference node using two key techniques: (a) prefix-aware request scheduling, and (b) predictive KVCache eviction.

5.1 Prefix-aware Request Scheduling

CACHEWISE leverages a prefix-aware request scheduler that prioritizes requests that can reuse KVCache already resident in XPU memory. Specifically, at time t , CACHEWISE selects request r_i which requires fewest additional blocks $a_i(t)$ to be allocated as the next request to dispatch.

Although greedy prefix-aware scheduling has been explored in prior systems such as SGLang, we observe that

it is particularly effective for coding agent workloads because it directly minimizes the *thrashing* effect on other sessions which will eventually re-use their evicted blocks as well. Furthermore, dispatching requests in order of $a_i(t)$ approximates *shortest-job-first* [17] scheduling which additionally minimizes overall system queueing at the expense of per-request responsiveness, since for closed-loop agent workloads, optimizing end-to-end session completions is more important than minimizing per-request TTFT or TBT, as no user is waiting on any individual request.

5.2 Predictive KVCache Eviction

While prefix-aware scheduling reduces the overhead of evictions, they remain inevitable because sessions keep accumulating context across turns and overall memory required increases. Ideally, the optimal eviction strategy follows Belady’s rule [7], i.e., at time t , evict blocks whose next access time $\tau_i(t)$ is furthest into the future, so that the evicted memory can be used for other sessions for the entire duration $\tau_i(t)$. Conversely, if the evicted blocks will be required almost immediately, and because $W(t) > M$, to allocate them again, blocks from some other session must be evicted at $\tau_i(t)$, which increases the overall eviction overhead incurred.

CACHEWISE proposes a predictive KVCache eviction policy that attempts to approximate the optimal eviction strategy by evicting blocks in decreasing order of $\tau_i(t)$. Our key insight is that although precisely predicting future KVCache reuse time for each session is challenging due to the variability in tool execution times, CACHEWISE only needs to predict the relative order of $\tau(t)$ across sessions to identify which blocks to evict. Furthermore, in-practice, we find that accurately identifying the session with the highest $\tau_i(t)$ is sufficient when using CACHEWISE, and lightweight estimators based on function names and arguments for each tool call can provide sufficient accuracy.

Predictor: For a session S_i that generated a tool call with metadata $m = (\text{tool_name}, \text{tool_args})$ at time T_i , our predictor estimates its next expected use time $\mathbb{E}[\tau_i(t)]$ using historical information, and uses these estimates to calculate the relative ordering to make its eviction decisions. Specifically, at time t , CACHEWISE estimates the expected remaining time for the tool call to complete, i.e., $\mathbb{E}[\tau_i(t) \mid \tau_i(t) - T_i > t -$

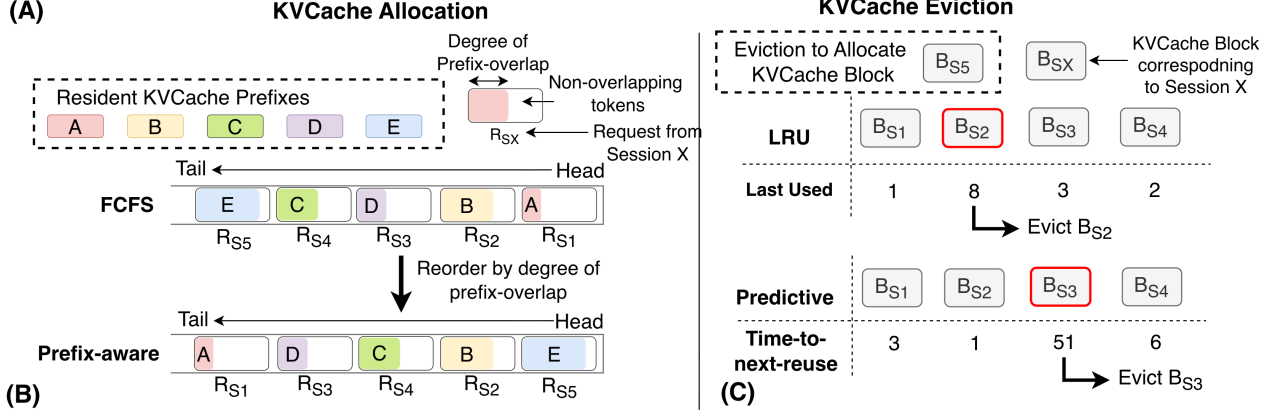
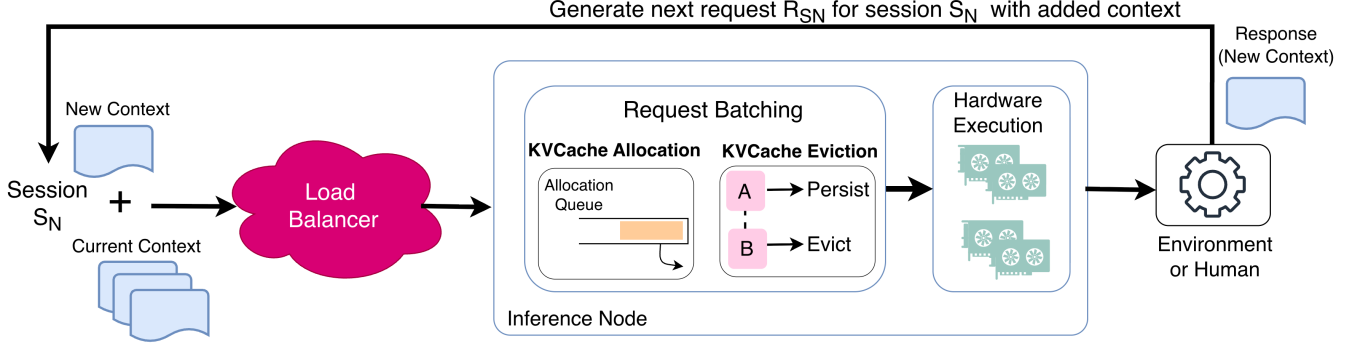


Figure 11. (A) End-to-end workflow of a session through a typical LLM serving system. (B) Prefix-aware request scheduling prioritizes requests by degree of overlap with KVCACHE resident on the XPU memory, leading to higher KVCACHE reuse. (C) Demonstration of predictive KVCACHE eviction choosing the block with the highest predicted reuse probability, rather than purely recency-based heuristics like LRU.

$T_i]$ by analyzing distributions of tool execution durations with similar metadata m from historical sessions with similar characteristics e.g., same user, same project.

Predictor Training: Figure 12 shows that relying solely on execution duration distributions of tools with the same names is insufficient to make these estimates accurately because execution times may vary significantly based on the tool arguments and the environment; for instance, *Bash* calls can have highly variable execution durations depending upon the arguments (e.g., Table 3) or the coding repository context (e.g., running *pytest* to run tests depends upon the specific test suite). To capture the effect of tool call arguments, CACHEWISE semantically clusters² historical samples based on the TF-IDF embeddings [30, 31] of their arguments, and uses these per-cluster distributions instead of per-tool distributions in such cases. TF-IDF encodes tool arguments into vectors by assigning higher weight to terms that are frequent within an argument but rare across other arguments³.

²We use KMeans [20] to generate clusters. We upper bound the number of clusters to a large value to achieve maximum granularity, allowing the model to distinguish fine-grained argument patterns within each tool type.

³Each tool argument is mapped to a TF-IDF vector of a maximum size of 5000 lexicons. We use scikit-learn’s implementation of ℓ_2 norm [30] to assign weights.

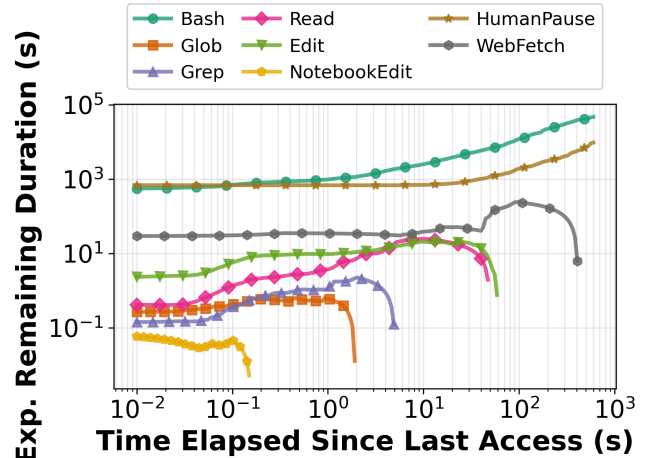


Figure 12. Distribution of tool execution times with respect to time elapsed since last access as observed in CATraces.

5.3 Implementation

We have implemented CACHEWISE using vLLM [21] in ~2,500 lines of Python code and extend the batch scheduler and KVCACHE manager to support CACHEWISE’s predictive block eviction mechanisms. Specifically, to implement CACHEWISE’s policies atop the vLLMs existing batch scheduling and KVCACHE management layer, we extend their

Command Pattern	P50	P90	P99	n
Python one-liners (<code>python -c</code>)	0.1s	0.2s	0.4s	9
Type checking (<code>mypy</code>)	10s	56s	182s	46
Docker builds (<code>docker compose</code>)	22s	89s	152s	30
Git operations (<code>git add / status</code>)	0.1s	34s	97s	34

Table 3. Bash tool call execution time distributions clustered by argument types. Columns P50, P90, and P99 show percentiles of tool execution times. n is the number of datapoints in the cluster.

KVCache block manager to associate additional session-level metadata with each KVCache block, i.e., `tool_name`, `tool_args` and T_i (time when the tool call was generated), which can be parsed by the vLLM engine itself.

When a request completes and its blocks become unreferenced (i.e., the reference count drops to zero), those blocks retain their metadata and are inserted into an eviction heap for ordered eviction. CACHEWISE replaces the default LRU eviction policy: when unreferenced blocks are added, our lightweight predictor quickly estimates the $\mathbb{E}[\tau_i(t)]$ based on the block’s attached S_i , and adds the block to the eviction heap with the prediction as its priority value. Note that CACHEWISE maintains predictions across blocks that share the same session metadata and multiple KVCache blocks belonging to the same session do not require invoking the predictor independently for each one.

However, as tool call executions progress, the predicted $\mathbb{E}[\tau_i(t)]$ value for a KVCache block becomes stale; as time elapses since the last access, the expected time-to-next-reuse shifts, and the stored prediction no longer reflects the current reuse likelihood of the block. To account for this, CACHEWISE periodically re-evaluates all unreferenced blocks through the predictor and rebuilds the eviction heap with updated $\mathbb{E}[\tau_i(t)]$ estimates. The rebuild frequency is controlled by a tunable parameter N_{rebuild} (in engine iterations): smaller values yield fresher estimates at higher CPU scheduling overhead, while larger values reduce overhead at the cost of stale predictions. In our evaluation, we set $N_{\text{rebuild}} = 3$ based on empirical observation that this balances prediction freshness and overhead.

6 Evaluation

We evaluate CACHEWISE using real multi-turn coding-agent traces sampled from the CATraces dataset. Our evaluation seeks to answer the following questions:

- How much CACHEWISE improve end-to-end performance compared to baselines?
- How much prefix-aware scheduling and predictive KVCache eviction contribute to end-to-end performance?
- Does CACHEWISE reduce KVCache data movement?
- Does CACHEWISE introduce non-negligible scheduling overheads?

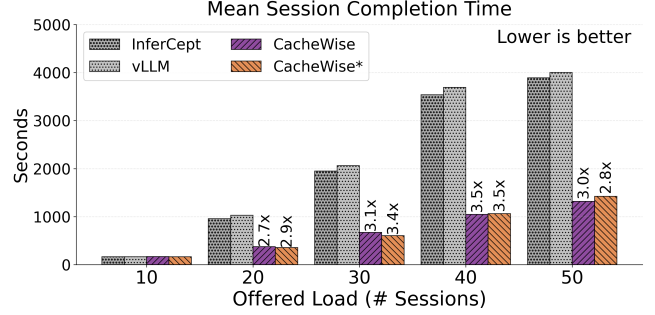


Figure 13. Session completion time (in seconds) for different KVCache management systems under the coding agent workload (sampled from CATraces).

6.1 Experimental Setup

Testbed: We ran experiments on a server with 2× H200 GPUs and AMD EPYC 9534 64-core CPU with Tensor Parallelism [34] (TP=2) enabled, with a combined GPU memory of 282GB. All experiments run with chunked-prefill enabled and we use the default value of max. 512 tokens per chunk [37].

Model: Similar to prior works [3, 46], we use a 32B parameter model, Qwen2.5-Coder-32B-Instruct [6], for our evaluation. Our prototype can execute any model, including models with grouped-query attention [4] or multi-query attention [32], without any modifications to model weights.

Workloads: We replay the traces collected in our dataset for our experiments deterministically. We randomly sample 80% of the sessions for *offline* training of the predictor, and use the remaining traces for our experiments. We assume that the system deploying CACHEWISE routinely logs request traces which can be used as the training data. In practice, trace distributions may drift over time as coding repositories and tool usage patterns evolve; in such cases, CACHEWISE’s estimators can be periodically retrained on recent traces to adapt to distribution shift, though we leave a thorough study of drift robustness to future work.

In our traces, users interactively respond to agent output to send a new request with the same KVCache prefix. Users can have long idle periods. During replay, we consider human idle periods as session boundaries. Our evaluation includes resumed sessions after human pause and these sessions have a significantly larger KVCache to prefill due to previous history. This is acceptable because it mimics our scheduler’s decisions: we always evict KVCache from idle sessions before evicting from sessions with toolcall induced delays.

Methodology: Our experiments launch N independent coding agent sessions concurrently on a single serving instance and runs until all sessions complete. We evaluate performance under increasing load. We repeat each experiment several times to ensure stability across runs.

Baselines: We compare CACHEWISE against: (a) vLLM [21]

with block-level LRU KVCache eviction and FCFS admission, (b) **InferCept** [2] with FCFS scheduling and moving-average-based eviction that predicts tool call durations from recent history. Although, we did not directly evaluate against SGLang, which already implements prefix-aware scheduling, our ablations (described below) cover this scenario as well.

Ablations: We also present (a) **CACHEWISE***, a version of **CACHEWISE** that has access to ground truth tool latencies to make evictions (b) **vLLM(Prefix-aware)** and (c) **InferCept(Prefix-aware)** which add prefix-aware scheduling to these baselines.

Metrics: We compare (1) token goodput (rate of new tokens computed per unit time), request throughput (LLM requests/second) (2) request completion time, (3) session completion time (the sum of LLM request latencies across a single session without the tool execution durations. We exclude tool execution durations as they are determined by the external environment and independent of the serving system), and (4) KVCache transfer volume (the amount of data moved due to evictions).

6.2 End-to-End Performance Improvements

Session-level performance: Figure 13 shows session-level performance across varying load levels. Observe that at lower load levels ($N \leq 10$), all systems exhibit similar performance, which is expected as there is negligible memory (KVCache) contention at low load levels, leaving sufficient GPU memory capacity for all sessions to execute concurrently. At higher load levels ($N > 10$), **CACHEWISE** consistently outperforms the baselines, achieving $2.7\times$ – $3.5\times$ lower completion time as compared to **vLLM** and **InferCept**. Further, **CACHEWISE** achieves performance comparable to the **CACHEWISE*** with ground-truth tool latencies, indicating that our lightweight estimators offer sufficient accuracy to identify which sessions blocks should be evicted. In some configurations, **CACHEWISE** marginally outperforms **CACHEWISE***, which we attribute to scheduling dynamics induced by differences in eviction decisions; evicting different blocks changes which requests can be scheduled with prefix-aware scheduling, which in turn affects the order and timing of requests being processed.

Figure 14 shows our evaluation of token goodput, KVCache block evictions, request-level latency, and throughput with respect to system load.

Token goodput: Observe that for $N > 10$, **CACHEWISE** consistently outperforms **vLLM** and **InferCept** with respect to token goodput ($1.64\times$ – $2\times$ improvement). **CACHEWISE** maximizes KVCache reuse, and consistently reduces the number of evicted KVCache blocks by 2 – $2.6\times$ compared to **vLLM** and **InferCept**, which enables it to increase the useful tokens generated per-unit time. Also observe that the token goodput decreases with increasing load for all systems. This is

expected, because at higher load more sessions compete for GPU memory, causing more KVCache evictions and recomputations for all systems. However, **CACHEWISE** minimizes the number of unnecessary evictions and recomputations, even at higher loads.

Request throughput: At higher load levels ($N > 10$), **CACHEWISE** consistently outperforms **vLLM** and **InferCept**. **CACHEWISE** achieves $1.5\times$ – $2\times$ higher throughput with respect to baselines. This is expected since higher token goodput is directly correlated with higher rate of request completion.

Request latency: **CACHEWISE** outperforms both **vLLM** and **InferCept** in terms of median (P50), and P90 request completion times across all load levels. At P50, **CACHEWISE** achieves a 13 – $14\times$ lower latencies as compared to the baselines, underscoring the benefit of **CACHEWISE**’s better resource management. We observe that **CACHEWISE** exhibits higher P99 request latencies, especially at higher load levels. This is due to **CACHEWISE**’s prefix-aware scheduling policy that defers new requests that lack history and is acceptable—session completion time is the appropriate user-facing metric, rather than request tail latency.

6.3 Performance Ablations

To better understand the impact of different optimizations in **CACHEWISE**, we compare it against **vLLM(Prefix-aware)** and **InferCept(Prefix-aware)** which use the same prefix-aware scheduling as in **CACHEWISE** but do not have access to the predictive KVCache eviction policy.

Impact of predictive KVCache management: Figure 15 shows the gains from predictive KVCache eviction, where **CACHEWISE** outperforms both respect to token goodput and session completion time, achieving $1.2\times$ – $1.6\times$ higher token goodput than baselines. Consequently, **CACHEWISE** also achieves $\sim 1.7\times$ – $2\times$ lower session completion time than **vLLM(Prefix-aware)** and **InferCept(Prefix-aware)**.

InferCept(Prefix-aware) computes tool execution durations by taking moving average of the prior tool executions in the session, disregarding the tool name, arguments, and the last accessed time of the KVCache blocks, leading to inaccurate time-to-next-reuse decisions, and hence suboptimal eviction decisions.

Impact of prefix-aware scheduling: Figure 16 shows the effect of prefix-aware scheduling applied independently to all baselines. Prefix-aware scheduling alone improves token goodput by $\sim 1.38\times$ – $1.64\times$ at $N = 30$ and $\sim 1.6\times$ – $1.7\times$ at $N = 40$ (Figure 16(a)), and reduces session completion time by $\sim 1.8\times$ – $2.35\times$ at $N = 30$ and $\sim 1.85\times$ – $2.66\times$ at $N = 40$ (Figure 16(b)). These gains hold even in the absence of accurate KVCache reuse predictions, reinforcing that the two optimizations are independent.

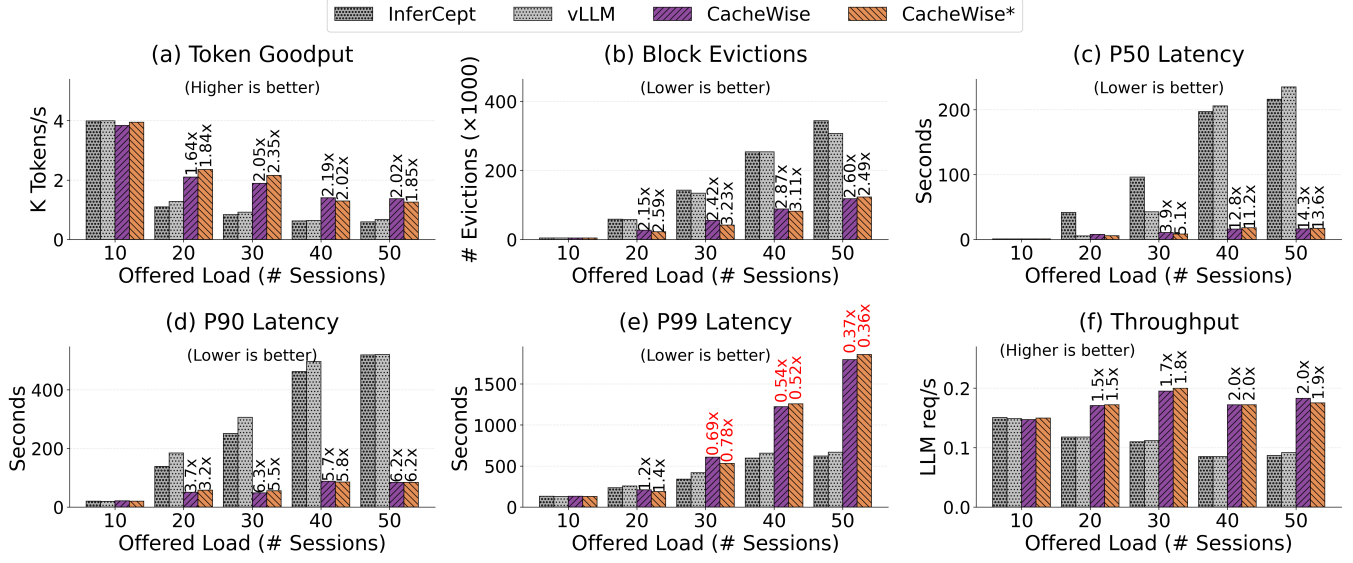


Figure 14. Serving efficiency of different KVCache management systems under the coding agent workload. (a) Goodput in-terms of the number of useful tokens generated per-unit time. (b)-(e) LLM request latency distribution (in seconds). (f) LLM request throughput (in number of requests completed per second).

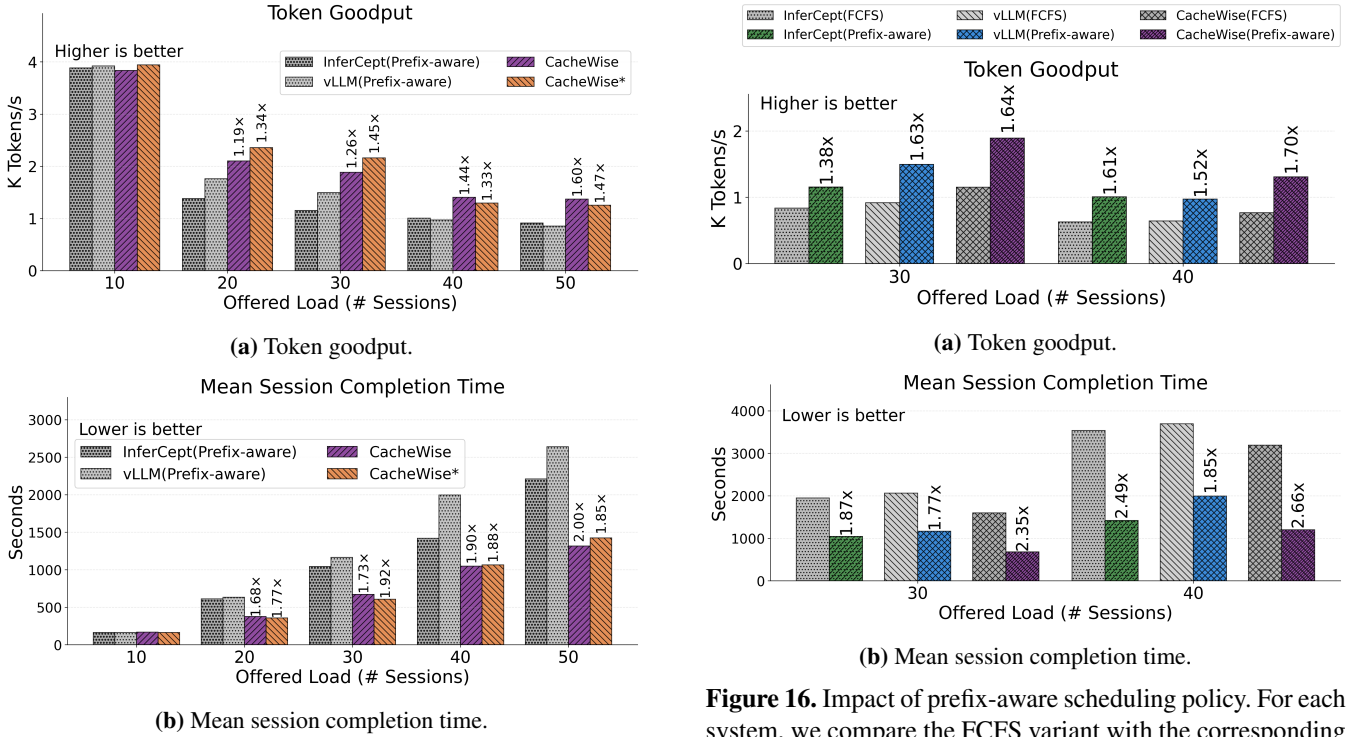


Figure 15. Impact of CACHEWISE's predictive KV-Cache eviction. vLLM(Prefix-aware), InferCept(Prefix-aware), CACHEWISE, and CACHEWISE* with prefix-aware scheduling enabled to isolate impact of KVCache eviction while keeping the scheduling policy fixed.

6.4 KVCache Movement

Modern LLM serving systems commonly offload evicted KVCache blocks to CPU memory or disk [10, 28] to avoid

Figure 16. Impact of prefix-aware scheduling policy. For each system, we compare the FCFS variant with the corresponding Prefix-aware variant.

recomputation. For such systems, CACHEWISE reduces the frequency of evictions by retaining reusable blocks rather than repeatedly evicting and recomputing them. These transfers consume significant interconnect bandwidths and can stall GPU execution while data is moved, reducing GPU utilization and token goodput.

To quantify this effect, we translate KVCache evictions

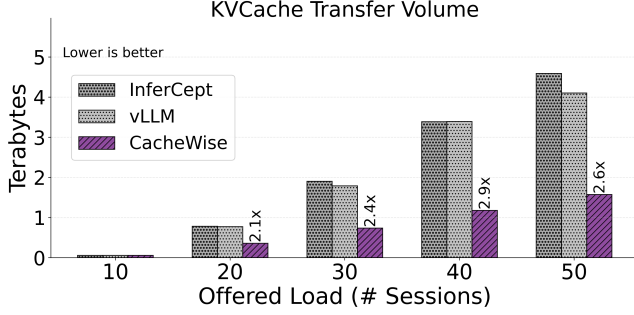


Figure 17. KVCACHE movement incurred (in TB of data) during the duration of each experiment, by translating eviction count to data size using KVCACHE block size.

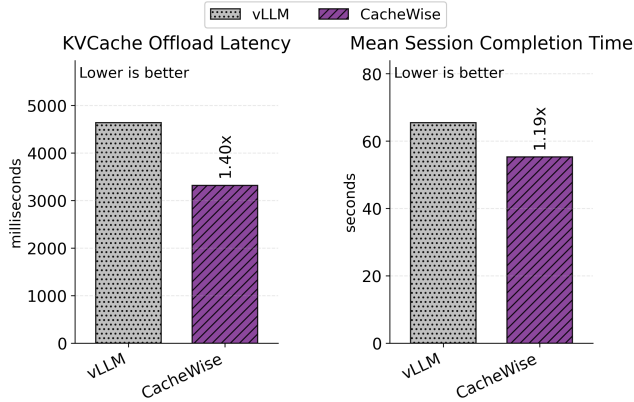
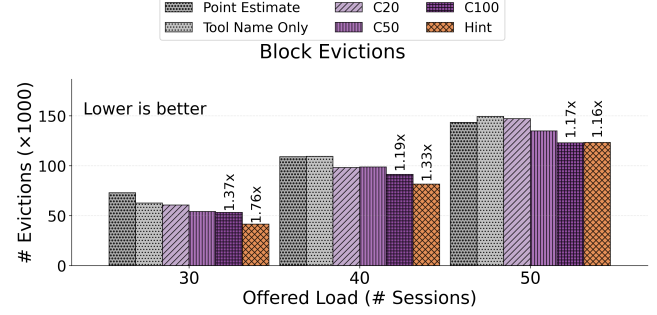


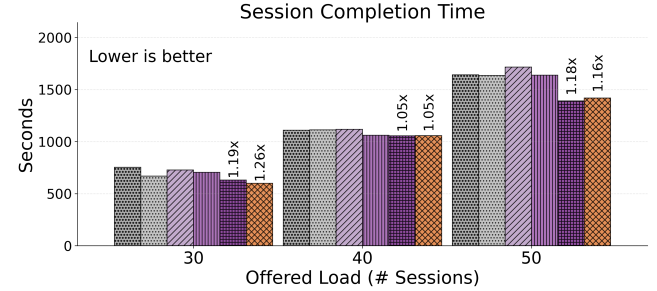
Figure 18. Overhead of KVCACHE offloading incurred due to transfer of KVCACHE from GPU–CPU and CPU–GPU over PCIe. The experiment was run with $N=30$ coding agent sessions sampled from CATraces. The KVCACHE offload latency (top-left) is the sum of swap-in and swap-out latencies observed by all the requests in the experiment.

into the corresponding volume of KVCACHE data transferred and report the cumulative transfer volume over the course of each experiment. Figure 17 shows the KVCACHE data transfer volume from our evaluation. CACHEWISE reduces KVCACHE transfer volume by $\sim 2\times$ – $2.6\times$ compared to baselines.

We further evaluate CACHEWISE on a system that physically moves KVCACHE blocks rather than recomputing them. In this configuration, evicted blocks are transferred from GPU memory to CPU DRAM and swapped back on demand when the corresponding session resumes, replacing recomputation overhead with idle times due to PCIe transfer latency. Figure 18 shows the KVCACHE offloading latencies and session completion time for $N=30$ concurrent sessions. CACHEWISE achieves $\sim 1.19\times$ lower session completion times than vLLM. The absolute gains under offloading are smaller than under recomputation. This is expected: PCIe transfers are substantially faster than prefill recomputation for large KVCACHE prefixes, so the penalty per eviction is lower, and the headroom for improvement is correspondingly reduced.



(a) Block Evictions.



(b) Mean session completion time.

Figure 19. Eviction count and session completion time in seconds. Lower values are better for both. **CX**=Tool argument based clustering with X clusters. **Hint**=Trace replay with ground truth tool durations.

6.5 Predictor Accuracy

We evaluate how prediction granularity affects eviction count and session completion time by progressively refining the estimator and measuring the resulting system efficiency. We consider the following predictors: **Point Estimates** uses a single point estimate of $\hat{\tau}$ derived from the global mean tool duration across all tool types and sessions. **Tool Name Only** conditions the estimate on tool name alone, using the per-tool empirical duration distribution (§5). **C20**, **C50**, and **C100** further refine the estimate by clustering tool calls based on tool arguments, using 20, 50, and 100 clusters respectively. Increasing the cluster count beyond 100 yielded no additional clusters, as the data did not support finer granularity. Using cluster **Hint** uses ground-truth tool duration from the trace. Figure 19 reports eviction counts and session completion times across four predictor variants and three load levels ($N = \{30, 40, 50\}$). We observe that as the prediction granularity increases from **Point Estimate** to **C100**, both eviction count and session completion time decrease across values of N . **C100** achieves the highest performance, with upto 19% improvement in session completion time.

6.6 Scheduling Overheads

Figure 20 compares the CPU-side scheduling overhead incurred by the CACHEWISE’s KVCACHE management policies against the GPU model execution time. The scheduling



Figure 20. CPU scheduling overheads (Sched.) vs. GPU execution overhead (Model Exec.) for different systems.

overhead corresponds to the time spent making scheduling and eviction decisions prior to each model iteration that executes on the GPU. CACHEWISE incurs $\sim 2.4\times$ – $3\times$ higher CPU scheduling overhead as those incurred by vLLM. However, the absolute CPU scheduling overheads are negligible when compared to the corresponding gains in lower model execution times. At $N = 40$, CACHEWISE incurs higher CPU scheduling overhead, increasing from 0.33 s (vLLM) to 0.99 s ($3.0\times$ increase). However, this overhead increase is more than offset by a significant reduction in GPU model execution time, which decreases from 16.6 s to 11.2 s ($1.48\times$ improvement). Overall, CACHEWISE yields a net reduction of ~ 4.7 s per request, demonstrating that savings from reduced model execution far outweigh the additional CPU scheduling cost. In relative terms, scheduling overhead accounts for roughly $\sim 6\%$ of total request time under vLLM and increases to about $\sim 9\%$ under CACHEWISE. However the absolute reduction in model execution time (~ 4.7 s for $N = 40$) is responsible for the significant gains in end-to-end performance.

7 Related Work

Serving systems for emerging LLM applications: Our work differs from recent works, e.g., Pie [15], Autellix [23], which create novel programming abstractions for agentic workloads. These systems tightly couple the agent implementation with the serving system and cannot work with existing widely deployed coding agents, while CACHEWISE can because it only modifies the serving system for these workload-specific characteristics.

Session-aware vs. request-aware scheduling: Existing serving systems including vLLM [21] and InferCept [2], typically schedule each request independently rather than optimizing for session completion times. Although SGLang [45] introduces prefix-aware scheduling to reuse KVCache across requests that share a common system prompt, their work primarily targets sharing across independent requests, while

our work demonstrates this is especially valuable for long-running, closed-loop coding agent sessions.

Predictive vs. recency-based eviction: vLLM, SGLang, and Mooncake [28] all rely on LRU eviction, which uses only past access recency and is oblivious to future reuse. InferCept [2] takes a step toward prediction by estimating tool call durations from a session-local moving average; however, this estimate is agnostic to tool type and arguments, limiting its accuracy for coding agent workloads where tool durations vary by orders of magnitude across tool types §5). InfiniGen [22] dynamically manages KVCache with speculative offloading but does not predict reuse from session metadata. Although predictive caching has been studied theoretically previously [24], CACHEWISE applies this practically for KVCache management.

KVCache tiered storage and memory management: LM-Cache [10] and Mooncake [28] address memory pressure by offloading evicted KVCache blocks to lower storage tiers (DRAM, SSD, or remote memory). NEO [18] offloads part of the attention computation and KVCache state to the CPU to increase effective batch size. Jenga [43] introduces a two-level memory allocator for heterogeneous KVCache embeddings, addressing memory fragmentation through LCM-based allocation. EvicPress [13] jointly applies lossy compression and adaptive eviction across storage tiers. Tiered storage is complementary to CACHEWISE’s approach. By reducing the frequency of evictions, CACHEWISE directly reduces the volume of data that must be moved across tiers, lowering the bandwidth and latency overhead that tiered systems must absorb.

8 Conclusion

Coding agents are emerging as an important class of LLM workloads, but they differ fundamentally from traditional chat workloads. Through a study of CATraces, our collected dataset of real-world coding-assistant sessions, we show that coding agents execute as long-running closed loops, generate predominantly tool-initiated turns, and repeatedly reuse large, growing prefixes. These properties make serving efficiency depend not only on per-request execution, but on how well the system preserves and reuses KVCache across turns within a session.

We present CACHEWISE, a KVCache management layer for coding-agent workloads. CACHEWISE combines prefix-aware scheduling, which prioritizes requests that can reuse resident KVCache, with predictive eviction, which retains prefixes expected to be reused soon based on tool execution metadata. Implemented in vLLM and evaluated on real coding-agent traces, CACHEWISE reduces KVCache evictions by up to 2 – $2.6\times$ and improves end-to-end session completion time by up to $\sim 3.5\times$.

References

- [1] 2025. Prefix Aware Routing — vLLM Production Stack. <https://docs.vllm.ai/projects/production-stack/en/vllm-stack-0.1.4/tutorials/prefixaware.html>. Accessed: 2026-03-11.
- [2] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiyang Zhang. 2024. INFERCEPT: Efficient Intercept Support for Augmented Large Language Model Inference. In *Forty-first International Conference on Machine Learning*. Vienna, Austria.
- [3] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming throughput-latency tradeoff in LLM inference with sarathi-serve. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation* (Santa Clara, CA, USA) (*OSDI'24*). USENIX Association, USA, Article 7, 18 pages.
- [4] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 4895–4901. doi:10.18653/v1/2023.emnlp-main.298
- [5] Anthropic. 2025. Claude Code — AI coding assistant. <https://www.claude.com/product/claude-code>. Accessed: 2026-03-11.
- [6] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, Dayiheng Liu, Gao Liu, Chengqiang Lu, Keming Lu, Jianxin Ma, Rui Men, Xingzhang Ren, Xuancheng Ren, Chuanqi Tan, Sinan Tan, Jianhong Tu, Peng Wang, Shijie Wang, Wei Wang, Shengguang Wu, Benfeng Xu, Jin Xu, An Yang, Hao Yang, Jian Yang, Shusheng Yang, Yang Yao, Bowen Yu, Hongyi Yuan, Zheng Yuan, Jianwei Zhang, Xingxuan Zhang, Yichang Zhang, Zhenru Zhang, Chang Zhou, Jingren Zhou, Xiaohuan Zhou, and Tianhang Zhu. 2023. Qwen Technical Report. arXiv:2309.16609 [cs.CL] <https://arxiv.org/abs/2309.16609>
- [7] L. A. Belady. 1966. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal* 5, 2 (1966), 78–101. doi:10.1147/sj.52.0078
- [8] BentoML. 2025. Prefix-aware Routing. <https://bentoml.com/llm/inference-optimization/prefix-aware-routing>
- [9] Amartya Chakraborty, Pareshe Dashore, Nadia Bathaee, Anmol Jain, Anirban Das, Shi-Xiong Zhang, Sambit Sahu, Milind Naphade, and Genta Indra Winata. 2025. T1: A Tool-Oriented Conversational Dataset for Multi-Turn Agentic Planning. arXiv:2505.16986 [cs.CL] <https://arxiv.org/abs/2505.16986>
- [10] Yihua Cheng, Yuhua Liu, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Kuntai Du, and Junchen Jiang. 2025. LMCACHE: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. arXiv:2510.09665 [cs.LG] <https://arxiv.org/abs/2510.09665>
- [11] Gregory Dexter, Shao Tang, Ata Fatahi Baarzi, Qingquan Song, Tejas Dharamsi, and Aman Gupta. 2025. LLM Query Scheduling with Prefix Reuse and Latency Constraints. arXiv:2502.04677 [cs.DS] <https://arxiv.org/abs/2502.04677>
- [12] Kuntai Du, Bowen Wang, Chen Zhang, Yiming Cheng, Qing Lan, Hejian Sang, Yihua Cheng, Jiayi Yao, Xiaoxuan Liu, Yifan Qiao, Ion Stoica, and Junchen Jiang. 2025. PrefillOnly: An Inference Engine for Prefill-only Workloads in Large Language Model Applications. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP)*. 399–414. doi:10.1145/3731569.3764834
- [13] Shaoting Feng, Yuhua Liu, Hanchen Li, Xiaokun Chen, Samuel Shen, Kuntai Du, Zhuohan Gu, Rui Zhang, Yuyang Huang, Yihua Cheng, Jiayi Yao, Qizheng Zhang, Ganesh Ananthanarayanan, and Junchen Jiang. 2025. EvicPress: Joint KV-Cache Compression and Eviction for Efficient LLM Serving. *arXiv preprint arXiv:2512.14946* (2025).
- [14] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. 2024. ServerlessLLM: Low-Latency Serverless Inference for Large Language Models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, 135–153.
- [15] In Gim, Zhiyao Ma, Seung seob Lee, and Lin Zhong. 2025. Pie: A Programmable Serving System for Emerging LLM Applications. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP)*. doi:10.1145/3731569.3764814
- [16] GitHub. 2021. GitHub Copilot. <https://github.com/features/copilot>. AI-powered code completion tool.
- [17] Mor Harchol-Balter, Alan Scheller-Wolf, and Andrew Young. 2009. Why segregating short jobs from long jobs under high variability is not always a win. In *2009 47th Annual Allerton Conference on Communication, Control, and Computing (Allerton)*. 121–127. doi:10.1109/ALLERTON.2009.5394853
- [18] Xuanlin Jiang, Yang Zhou, Shiyi Cao, Ion Stoica, and Minlan Yu. 2025. NEO: Saving GPU Memory Crisis with CPU Offloading for Online LLM Inference. In *Proceedings of Machine Learning and Systems (MLSys)*.
- [19] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world GitHub Issues?. In *International Conference on Learning Representations (ICLR)*.
- [20] KMeans. 2026. KMeans Scikit Learn. <https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html>.
- [21] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) (*SOSP '23*). Association for Computing Machinery, New York, NY, USA, 611–626. doi:10.1145/3600006.3613165
- [22] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. 2024. InfiniGen: Efficient Generative Inference of Large Language Models with Dynamic KV Cache Management. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, 155–172.
- [23] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E. Gonzalez, and Ion Stoica. 2025. Autellix: An Efficient Serving Engine for LLM Agents as General Programs. arXiv:2502.13965 [cs.LG] <https://arxiv.org/abs/2502.13965>
- [24] Thodoris Lykouris and Sergei Vassilvitskii. 2021. Competitive Caching with Machine Learned Advice. *J. ACM* 68, 4 (2021), 1–25. doi:10.1145/3447579
- [25] OpenAI. 2023. OpenAI Codex. <https://github.com/openai/codex>. GitHub repository, accessed January 24, 2026.
- [26] Pratyush Patel, Esha Choukse, Chaojie Zhang, Í nigo Goiri, Aashaka Shah, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*. 118–132. doi:10.1109/ISCA59077.2024.00019
- [27] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2023. Gorilla: Large Language Model Connected with Massive APIs. arXiv:2305.15334 [cs.CL] <https://arxiv.org/abs/2305.15334>
- [28] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. USENIX Association, Santa Clara, CA, 155–170. <https://www.usenix.org/conference/fast25/presentation/qin>
- [29] RyokoAI. 2023. ShareGPT 52K Dataset. <https://huggingface.co/>

- datasets/RyokoAI/ShareGPT52K. Accessed: 2026-03-11.
- [30] scikit-learn. 2026. Scikit-learn TF-IDF documentation. https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html.
 - [31] scikit-learn developers. 2024. TfidfVectorizer. https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html. Accessed: 2026-01-22.
 - [32] Noam Shazeer. 2019. Fast Transformer Decoding: One Write-Head is All You Need. arXiv:1911.02150 [cs.NE] <https://arxiv.org/abs/1911.02150>
 - [33] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.
 - [34] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2020. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. arXiv:1909.08053 [cs.CL] <https://arxiv.org/abs/1909.08053>
 - [35] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumnix: Dynamic Scheduling for Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, 173–191.
 - [36] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2023. Attention Is All You Need. arXiv:1706.03762 [cs.CL] <https://arxiv.org/abs/1706.03762>
 - [37] vLLM. 2026. vLLM Chunked Prefill. <https://docs.vllm.ai/en/v0.4.2/models/performance.html>. Default chunked-prefill size in vLLM.
 - [38] Zhangchen Xu, Adriana Meza Soria, Shawn Tan, Anurag Roy, Ashish Sunil Agrawal, Radha Poovendran, and Rameswar Panda. 2025. TOUCAN: Synthesizing 1.5M Tool-Agent Data from Real-World MCP Environments. arXiv:2510.01179 [cs.LG] <https://arxiv.org/abs/2510.01179>
 - [39] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R. Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*.
 - [40] John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. 2025. SWE-smith: Scaling Data for Software Engineering Agents. arXiv:2504.21798 [cs.SE] <https://arxiv.org/abs/2504.21798>
 - [41] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.
 - [42] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, Carlsbad, CA, 521–538. <https://www.usenix.org/conference/osdi22/presentation/yu>
 - [43] Chen Zhang, Kuntai Du, Shu Liu, Woosuk Kwon, Xiangxi Mo, Yufeng Wang, Xiaoxuan Liu, Kaichao You, Zhuohan Li, Mingsheng Long, Jidong Zhai, Joseph Gonzalez, and Ion Stoica. 2025. Jenga: Effective Memory Management for Serving LLM with Heterogeneity. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP)*. 446–461. doi:10.1145/3731569.3764823
 - [44] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. AutoCodeRover: Autonomous Program Improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 1592–1604. doi:10.1145/3650212.3680384
 - [45] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. arXiv:2312.07104 [cs.AI] <https://arxiv.org/abs/2312.07104>
 - [46] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, 193–210.